

1 Exploratory Data Analysis

The dataset consists of grayscale images (28×28 pixels). The training set contains 9,000 images.

1.1 Target Definitions

The dataset involves 6 specific shape types:

Squares, Circles, Up-Triangles, Right-Triangles, Down-Triangles, and Left-Triangles.

Classification Logic (135 Classes): The 135 classification categories are not arbitrary; they represent specific combinations of shape pairs. The dataset is constructed such that:

1. Two distinct shape types are selected from the 6 available.
2. The counts of these two shapes (a_i, a_j) always sum to 10 (e.g., 3 squares + 7 circles).
3. There are $\binom{6}{2} = 15$ pairs of shapes, and for each pair, there are 9 valid count combinations ($1+9$ to $9+1$). This results in $15 \times 9 = 135$ unique classes.

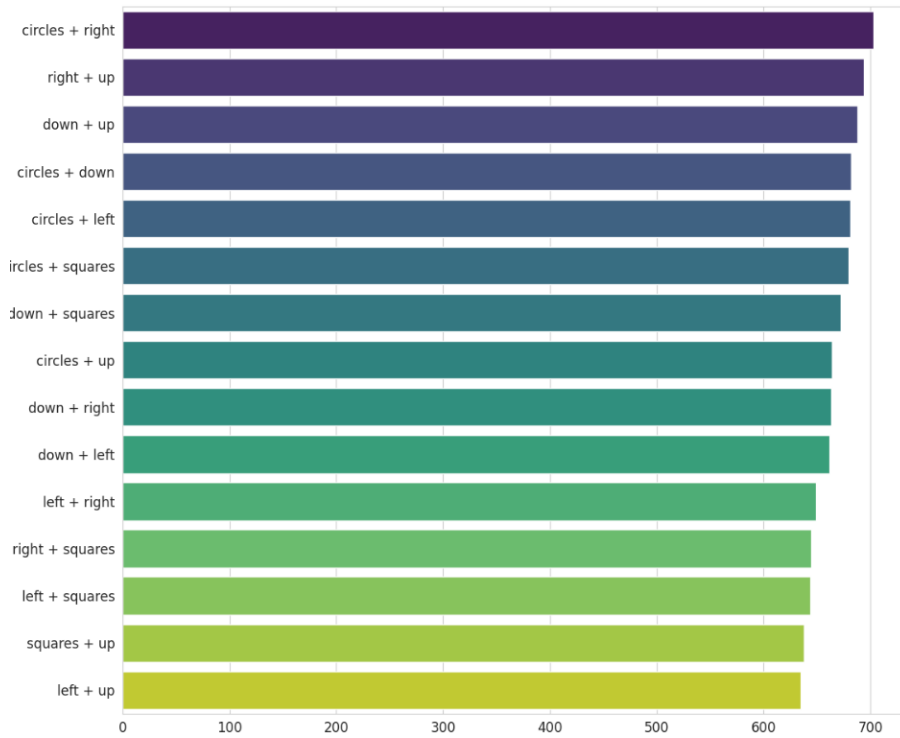


Figure 1: Distribution of the 15 pairs of shapes. We can see that data is relatively balanced.

2 Model Architecture

We implemented a custom CNN backbone with split heads for classification and regression.

2.1 Backbone

- **Conv1:** $1 \rightarrow 8$ channels, 3×3 kernel.
- **Conv2:** $8 \rightarrow 16$ channels, 3×3 kernel.
- **Conv3:** $16 \rightarrow 32$ channels, 3×3 kernel.
- **Conv4:** $32 \rightarrow 64$ channels, 3×3 kernel.
- **Flatten:** Output vector size $64 \times 28 \times 28 = 50,176$.
- **Dense:** Linear projection to 256 features.

2.2 Task Heads

- **Classification Head:** A dropout layer ($p = 0.7$) followed by a Linear layer mapping $256 \rightarrow 135$ with LogSoftmax.
- **Regression Head:** Linear layer mapping $256 \rightarrow 6$.

3 Data Augmentation

3.1 Strategy

- **Rotations ($90^\circ, 180^\circ, -90^\circ, -180^\circ$):** When the image is rotated, the ground truth count vector is permuted. For example, a 90° rotation shifts the count at index 2 ("Up") to index 3 ("Right").
- **Flips (Horizontal/Vertical):** Similar logic applies; a vertical flip swaps the counts for "Up" and "Down" triangles.

3.2 Justification

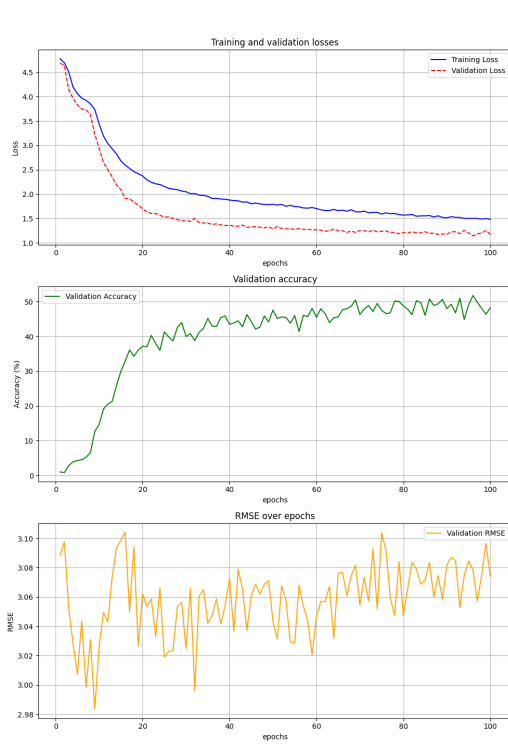
I used rotations and flips since they alter the images the most. The main issue I had was overfitting, so I wanted the data to be different almost every time the network trained. The other augmentations didn't help much with this issue during testing, so I omitted them in the end.

4 Results

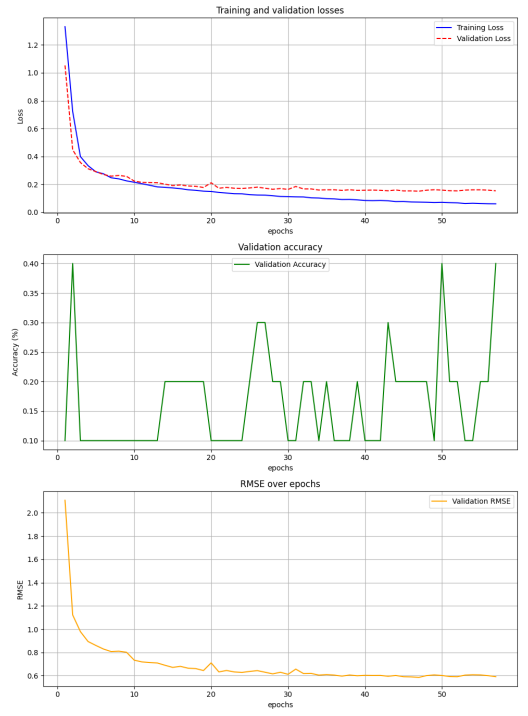
Table 1: Experimental Results (100 Epochs, Patience=10)

Exp	Configuration	λ_{cls}	λ_{cnt}	Loss	Acc (%)	RMSE
1	Classification Only	1.0	0.0	1.1426	51.80	-
2	Regression Only	0.0	1.0	0.1531	-	0.5853
3	Multi-Task	1.0	1.0	1.3545	50.50	0.6156

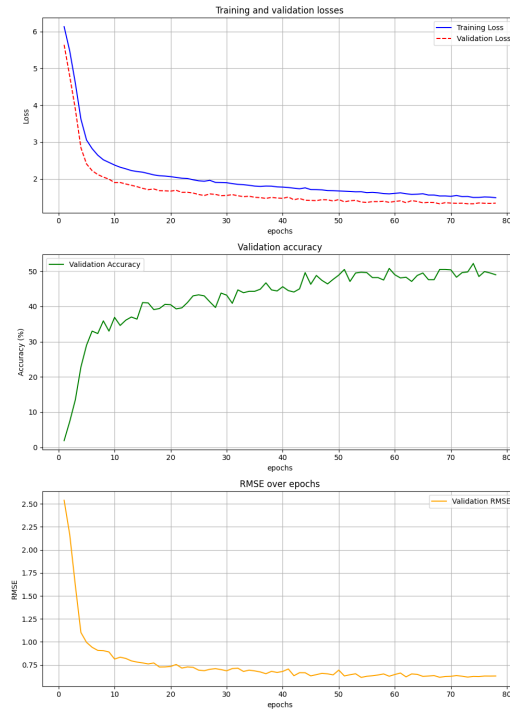
5 Learning Curves



(a) Classification-only: $\lambda_{cnt} = 0$



(b) Regression-only: classification loss ignored



(c) Multitask: $\lambda_{cnt} = 1$

Figure 2: Comparison of learning curves across three experiments.

6 Discussion

The regression problem turned out to be very easy for this backbone. The only change needed in the head was replacing 256 features with 6 features. However, the classification problem was much harder. I had to use very high dropout to fight overfitting, and even then, the classification loss stayed around 1.0, while the count loss dropped to 0.1. This shows that counting is very easy for this model to learn, and it neither helps nor harms the overall model when both tasks are trained together. Also, the λ_{cnt} parameter in the multi-task setup doesn't really matter (unless it is very large). This is because the count loss drops to around 0.1 very quickly (in about 20 epochs), so it stops affecting the total loss almost immediately.